EXPERIMENT NO: 04

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts

THEORY:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- uint / int: unsigned and signed integers of different sizes (e.g., uint256, int128).
- bool: represents logical values (true or false).
- address: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- bytes / string: store binary data or textual data.

Variables in Solidity can be state variables (stored on the blockchain permanently), local variables (temporary, created during function execution), or global variables (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- pure: cannot read or modify blockchain state; they work only with inputs and internal computations.
- view: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- Function Visibility defines who can access a function:
  - o public: available both inside and outside the contract.
  - o private: only accessible within the same contract.
  - o internal: accessible within the contract and its child contracts.
  - o external: can be called only by external accounts or other contract
- Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).

● Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage**: Data stored permanently on the blockchain. Examples: state variables.
- **memory**: Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata**: A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

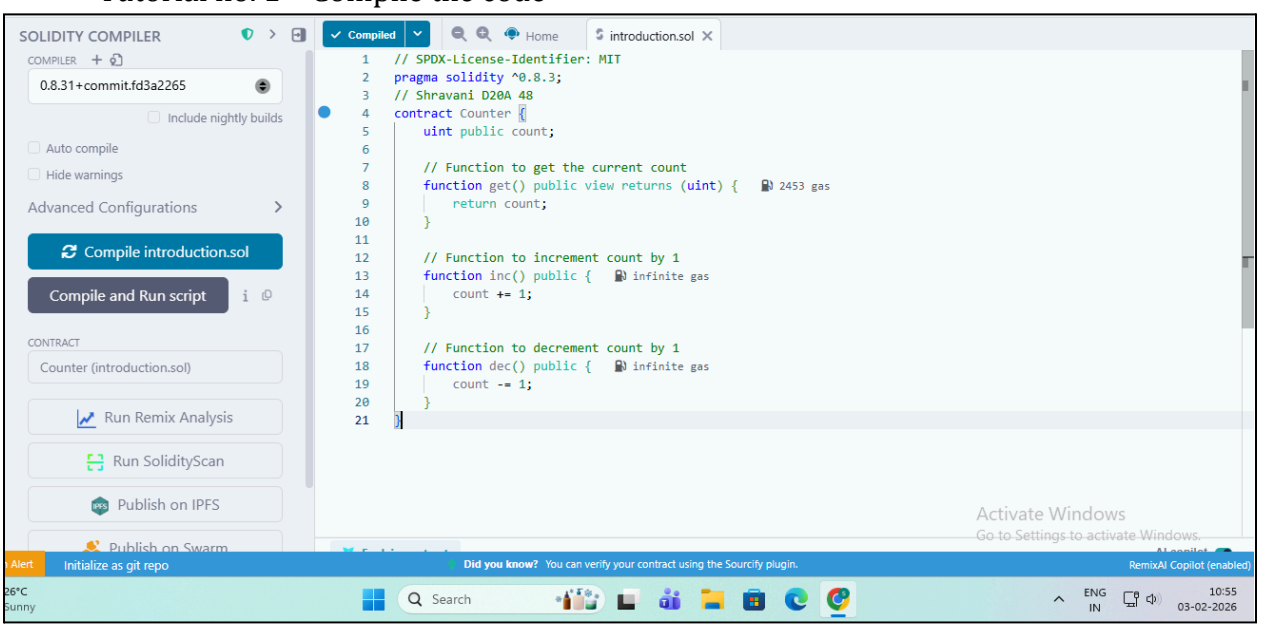
7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei**: Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price**: Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions**: Transactions are used for transferring Ether or interacting with contracts. Functions like transfer() and send() are commonly used, while call() provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

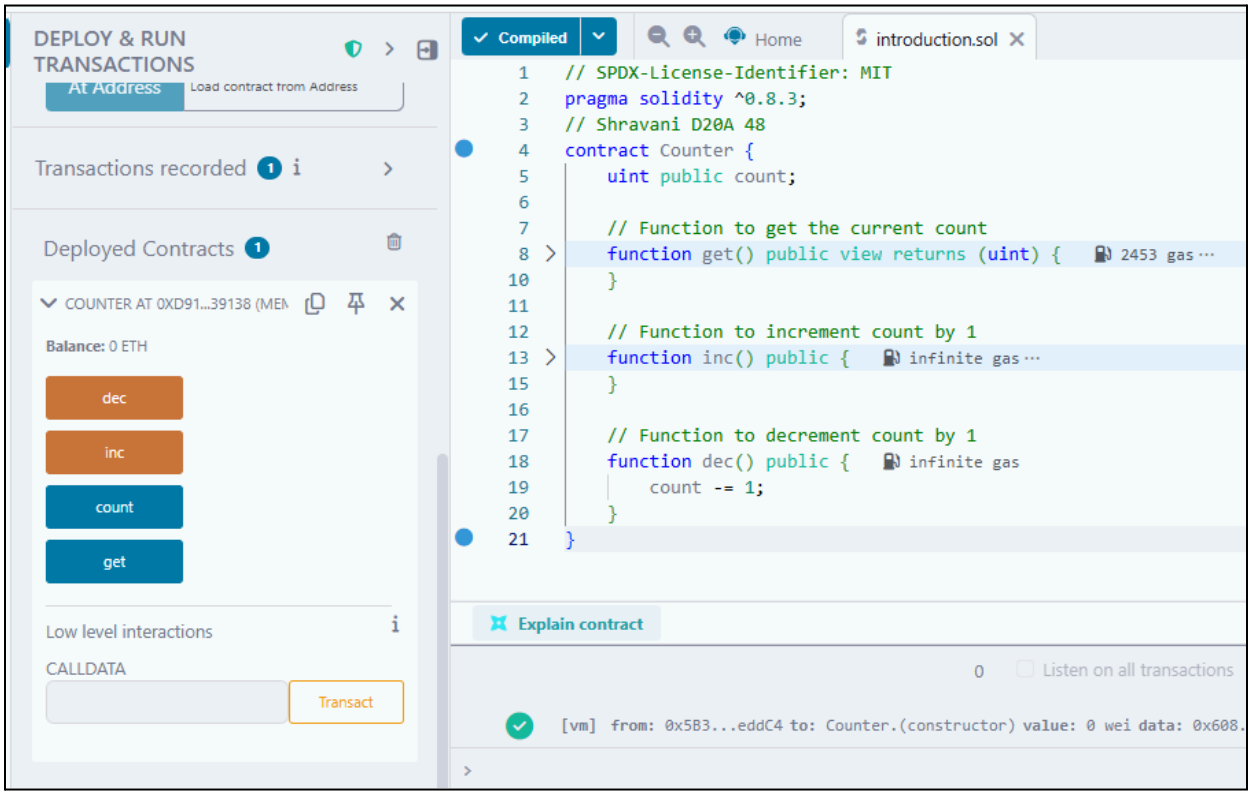
OUTPUT:

Tutorial 1:

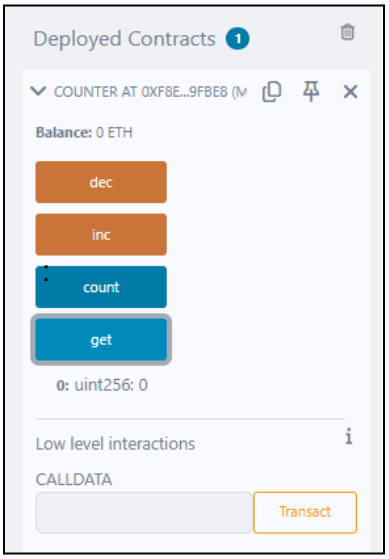
- Tutorial no. 1 – Compile the code



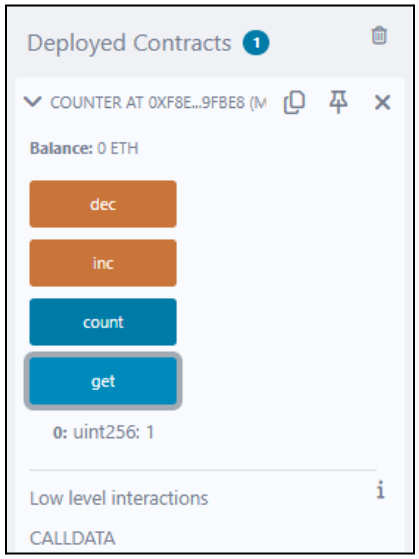
- Deploy the contract



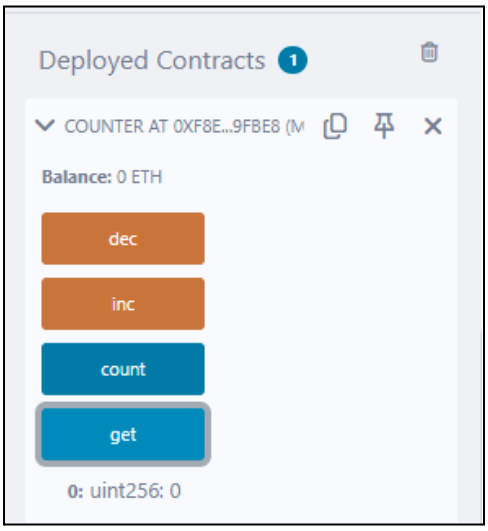
GET



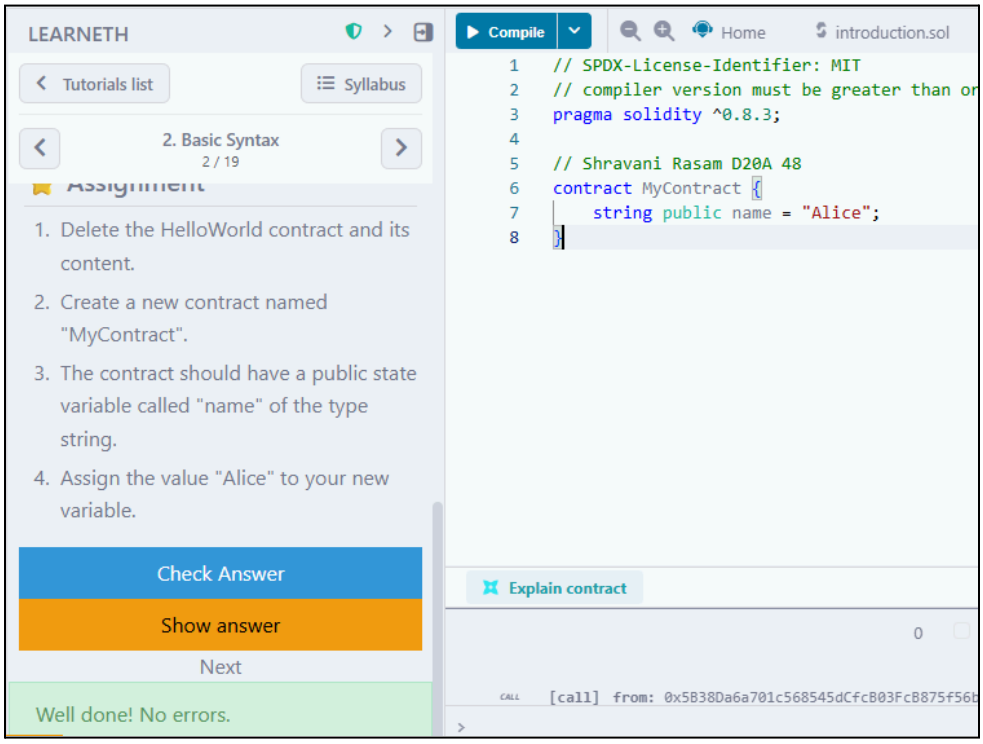
Increment



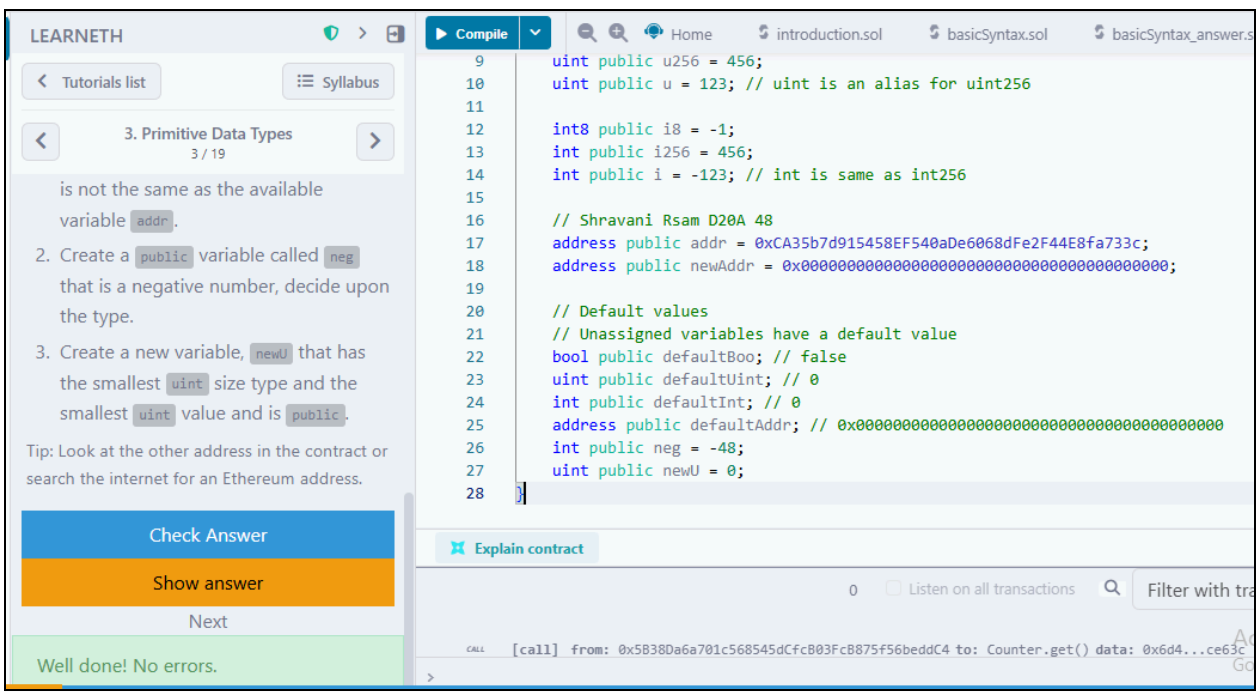
Decrement



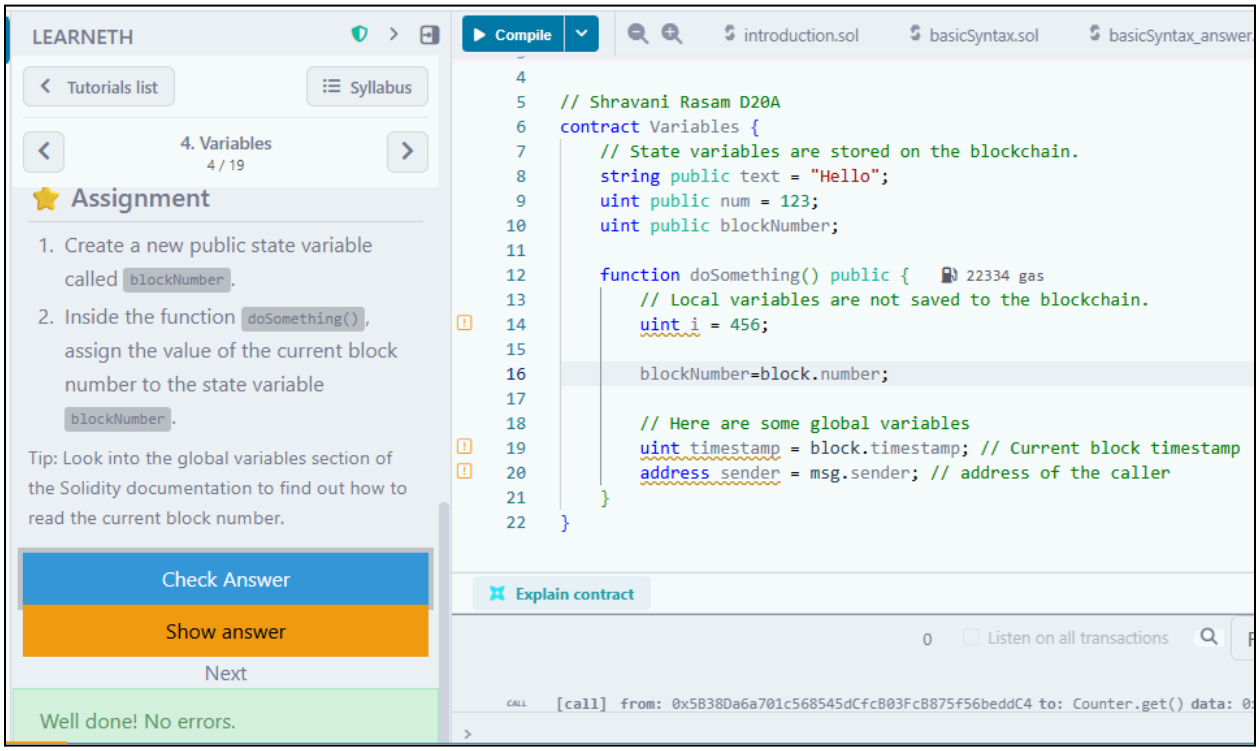
Tutorial 2:



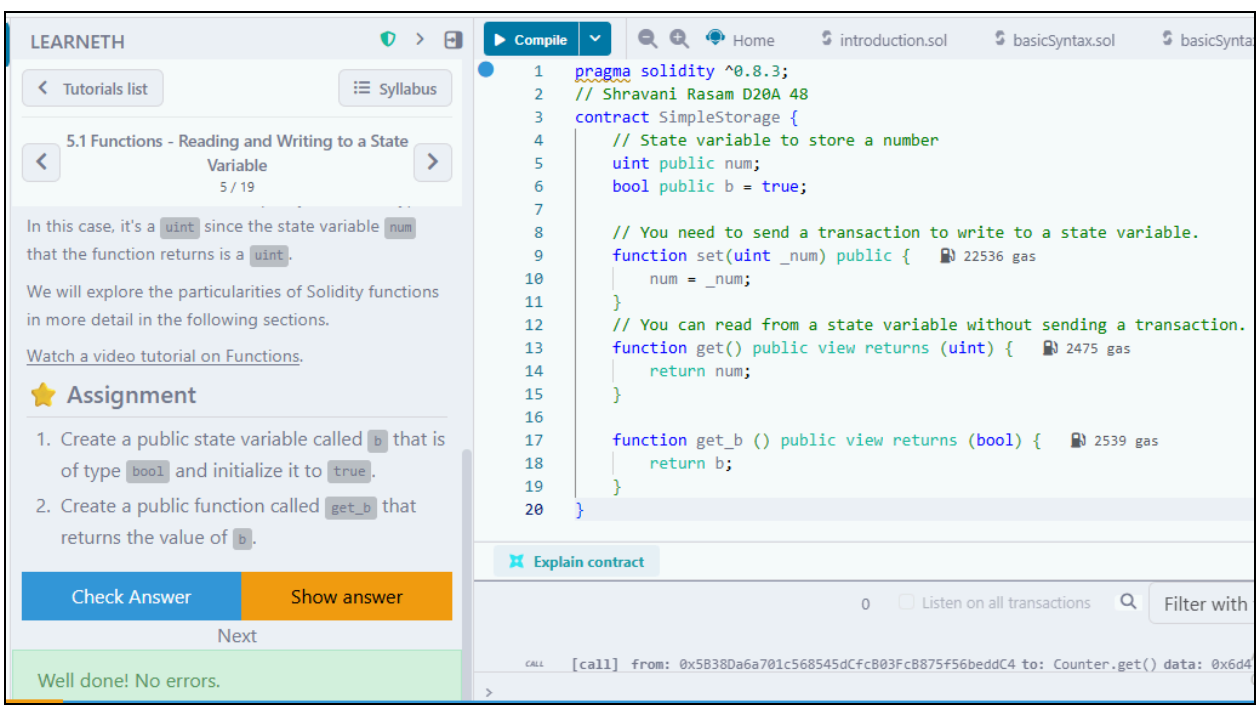
Tutorial 3:



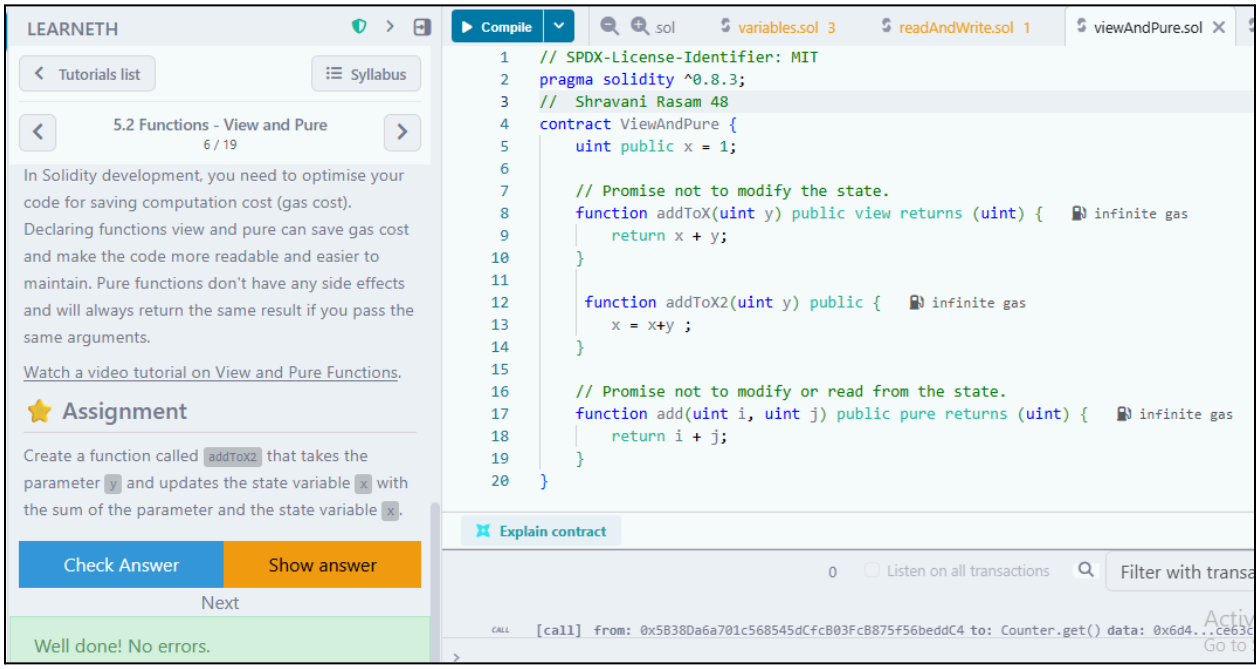
Tutorial 4:



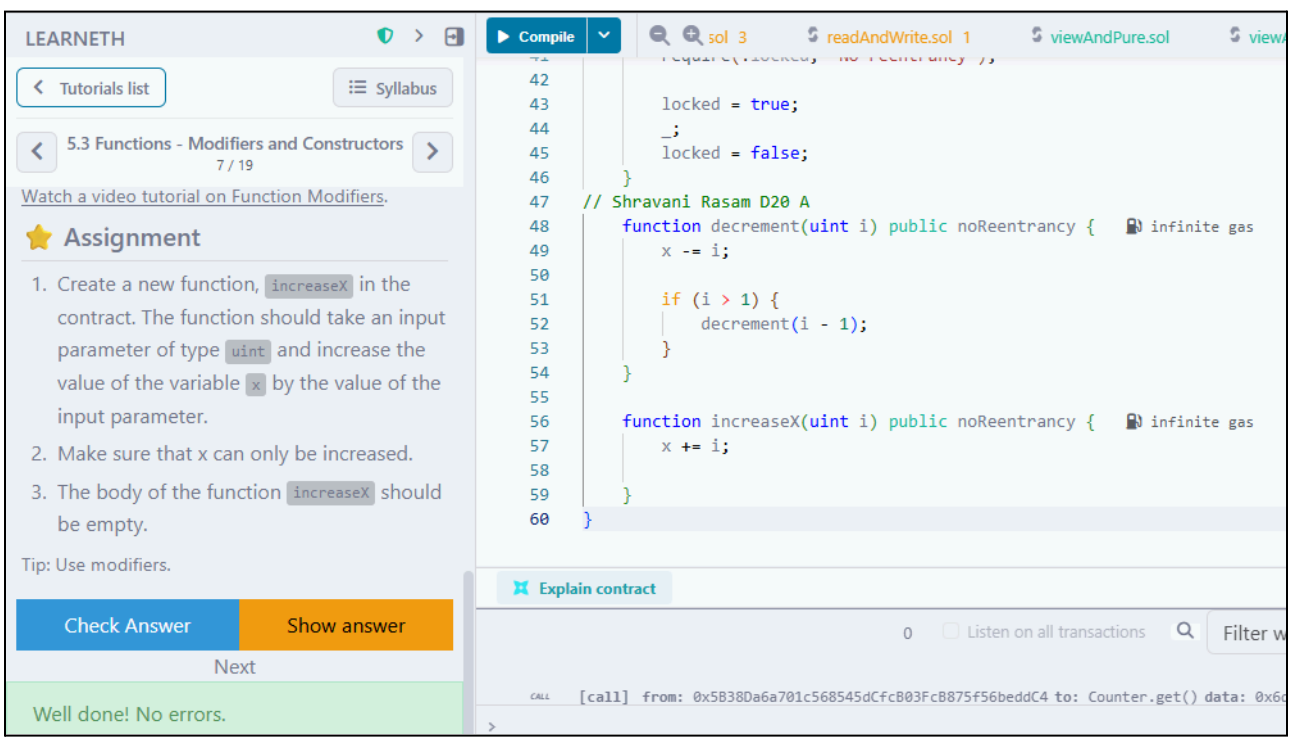
Tutorial 5:



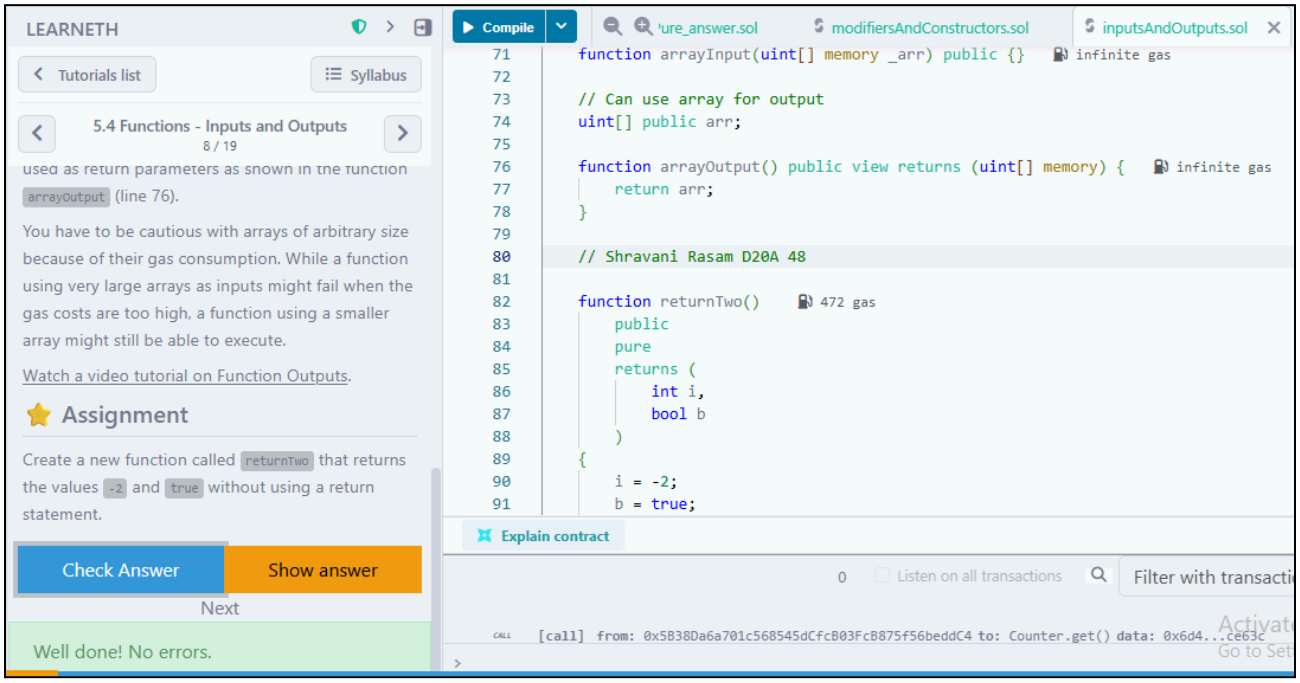
Tutorial 6:



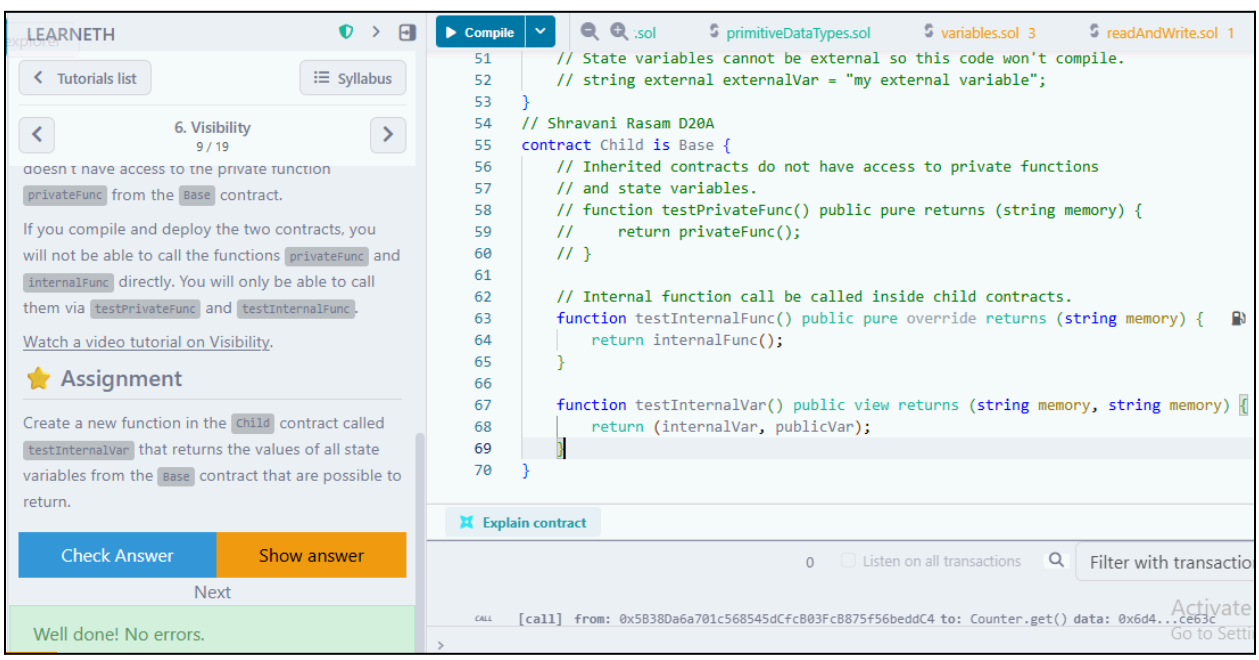
Tutorial 7:



Tutorial 8:



Tutorial 9:



Tutorial 10:

Tutorials list | Syllabus

7.1 Control Flow - If/Else
10 / 19

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternery operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer | Show answer

Next

Well done! No errors.

```
            return 1;
        } else {
            return 2;
        }
    }
    // Shravani Rasam D20A 48

    function ternary(uint _x) public pure returns (uint) {    infinite gas
        // if (_x < 10) {
        //     return 1;
        // }
        // return 2;

        // shorthand way to write if / else statement
        return _x < 10 ? 1 : 2;
    }

    function evenCheck(uint y) public pure returns (bool) {    infinite gas
        return y%2 == 0 ? true : false;
    }
}
```

Explain contract

0 Listen on all transactions Filter with t

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4...

Tutorial 11:

LEARNETH

Tutorials list | Syllabus

7.2 Control Flow - Loops
11 / 19

contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer | Show answer

Next

Well done! No errors.

Compile | readAndWrite.sol 1 | visibility.sol | ifElse.sol

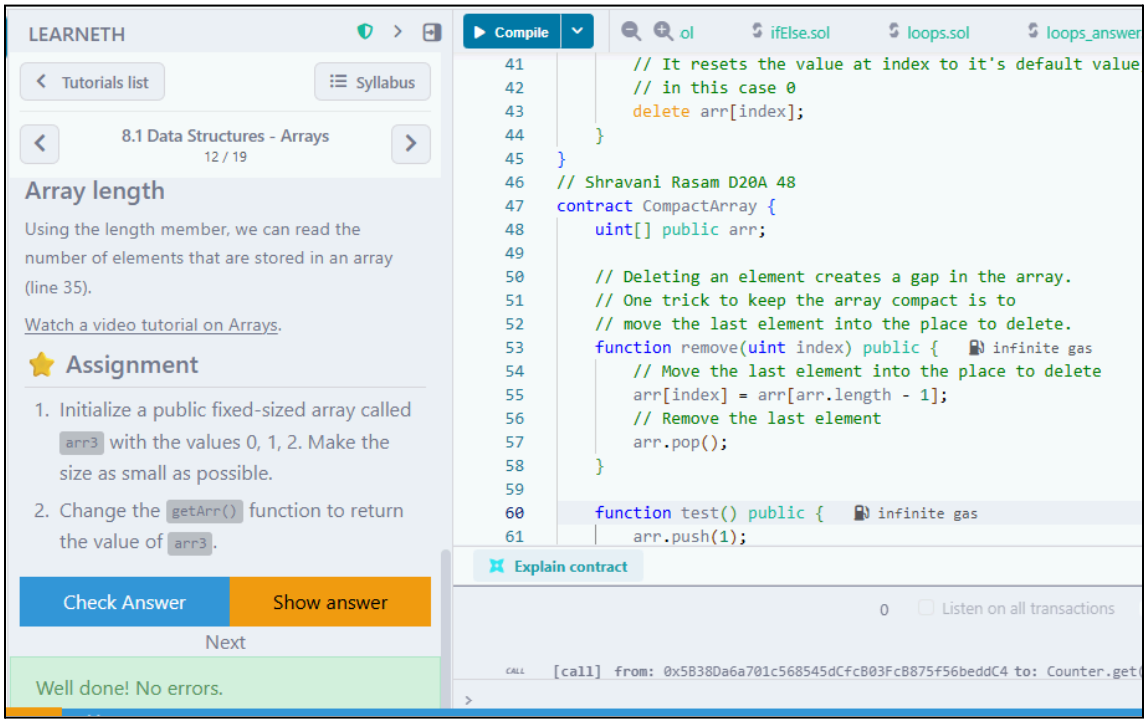
```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  // Shravani Rasam D20A
5  contract Loop {
6      uint public count;
7      function loop() public{    infinite gas
8          // for loop
9          for (uint i = 0; i < 10; i++) {
10             if (i == 5) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18             count++;
19         }
20
21         // while loop
```

Explain contract

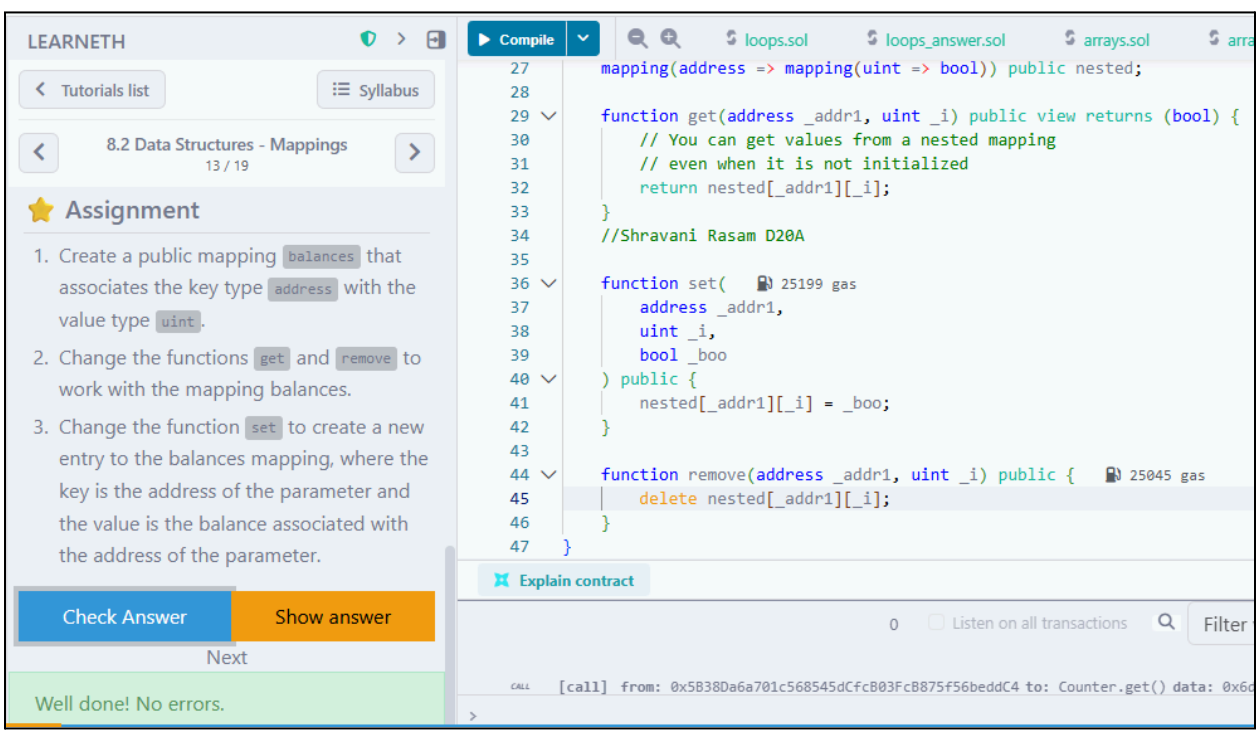
0 Listen on all transactions

[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() da

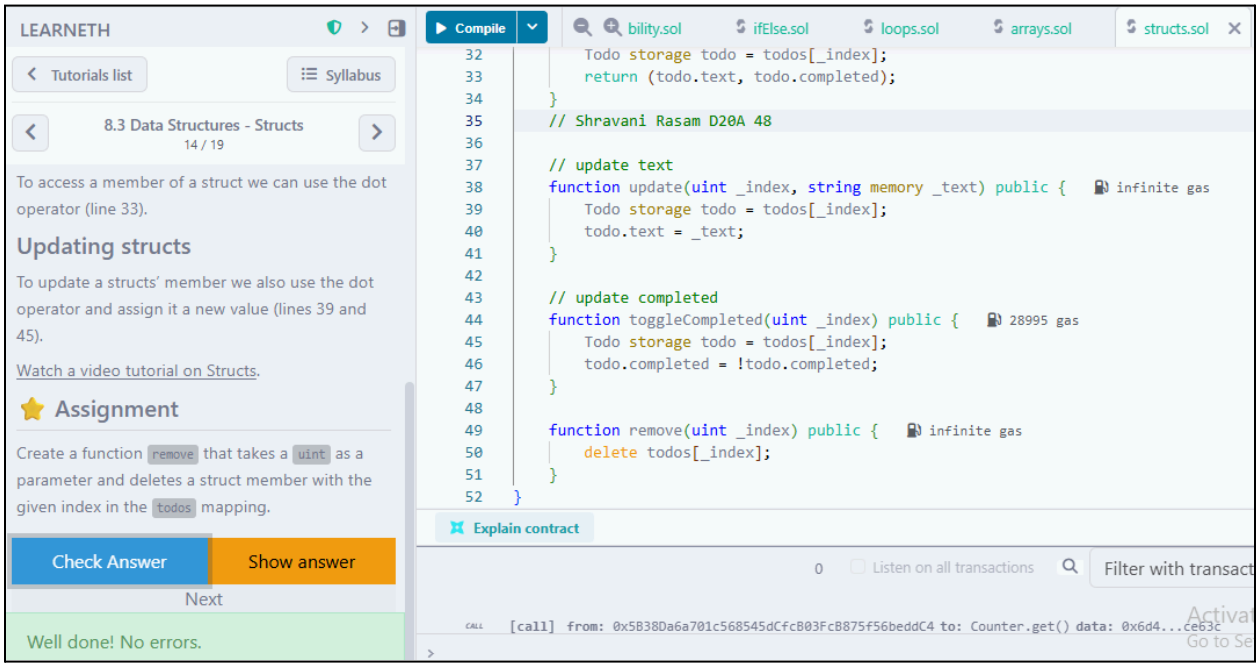
Tutorial 12:



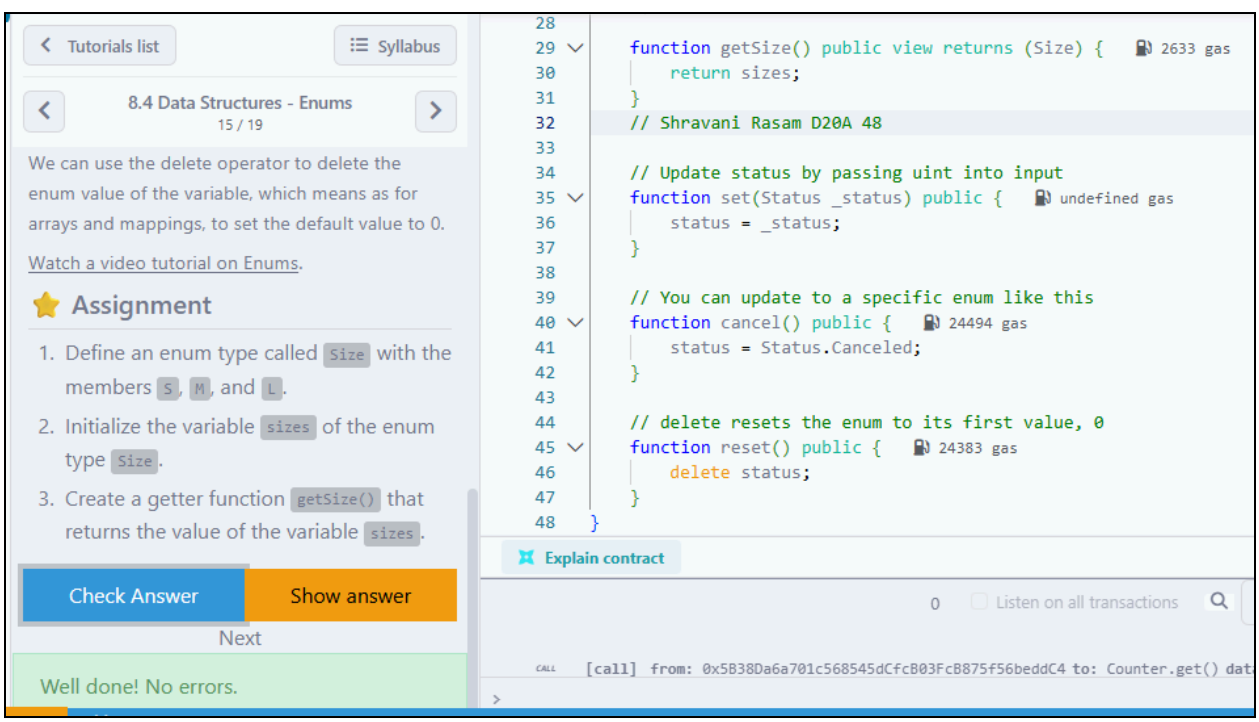
Tutorial 13:



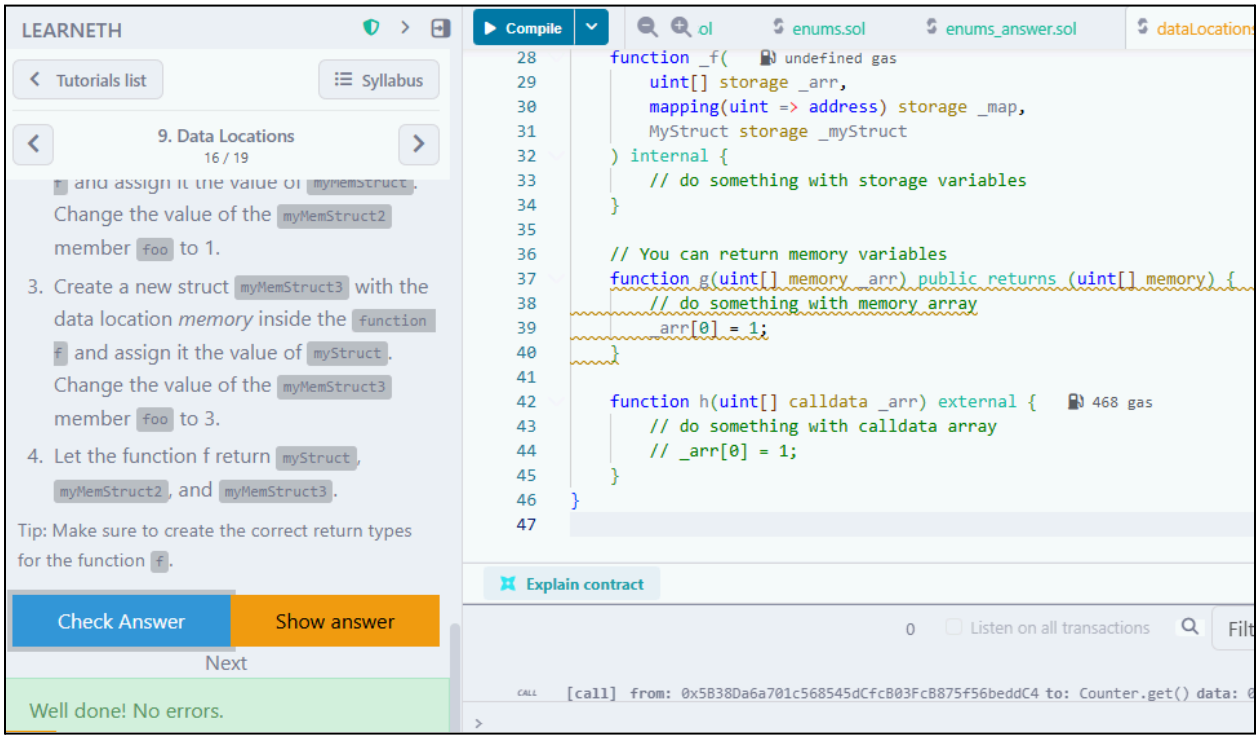
Tutorial 14:



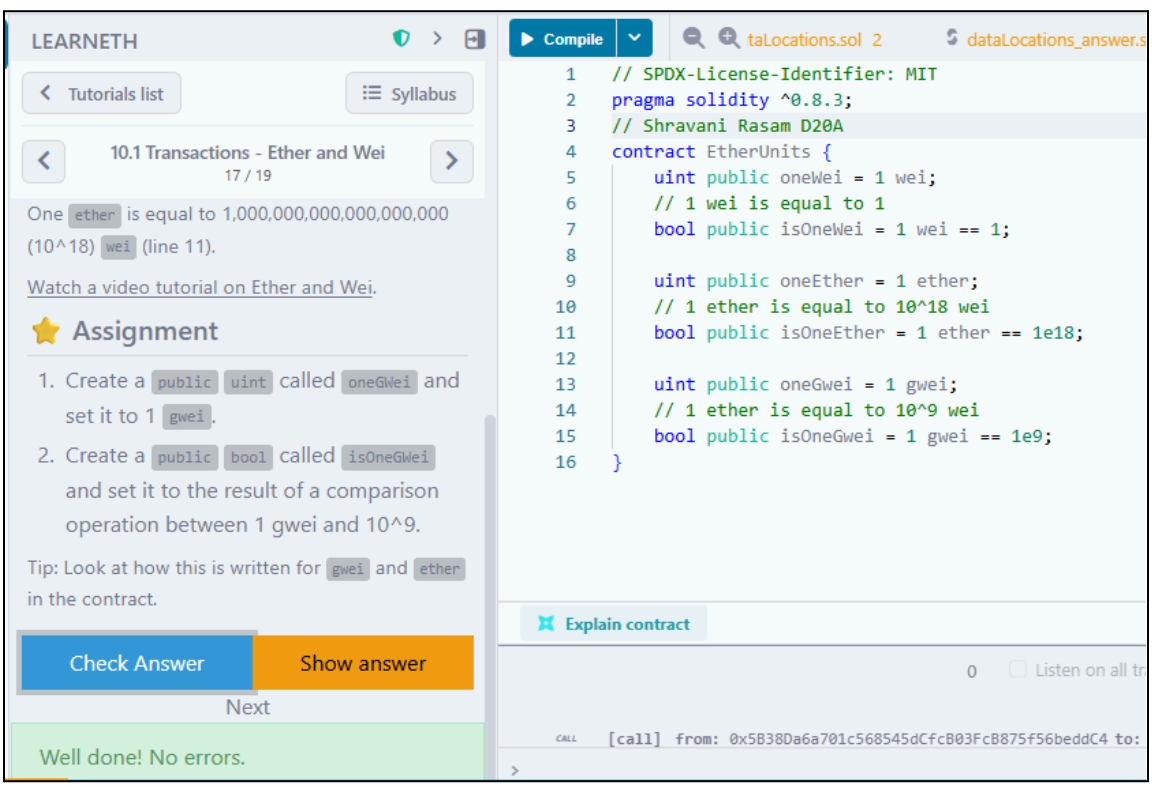
Tutorial 15:



Tutorial 16:



Tutorial 17:



Tutorial 18 :

LEARNETH

Tutorials list | Syllabus

10.2 Transactions - Gas and Gas Price
18 / 19

Learn more about *gas* on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer | Show answer

Next

Well done! No errors.

Compile | therAndWei.sol | etherAndWei_answer.sol | gasAndGasPrice.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
// Shravani Rasam D20A 48
contract Gas {
    uint public i = 0;
    uint public cost = 170367;

    // Using up all of the gas that you send causes your transaction t
    // State changes are undone.
    // Gas spent are not refunded.
    function forever() public {    infinite gas
        // Here we run a loop until all of the gas are spent
        // and the transaction fails
        while (true) {
            i += 1;
        }
    }
}
```

Explain contract

0 Listen on all transactions Filte

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x

Tutorial 19:

LEARNETH

Tutorials list | Syllabus

10.3 Transactions - Sending Ether
19 / 19

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type address.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer | Show answer

Next

Well done! No errors.

Compile | ndGasPrice.sol | gasAndGasPrice_answer.sol | sendingEther.sol

```
        require(sent, "Failed to send Ether");
    }
}
// Sheavani Rasam D20A 48
contract Charity {
    address public owner;

    constructor() {    165452 gas 141000 gas
        owner = msg.sender;
    }

    function donate() public payable {}    141 gas

    function withdraw() public {    infinite gas
        uint amount = address(this).balance;

        (bool sent, bytes memory data) = owner.call{value: amount}("");
        require(sent, "Failed to send Ether");
    }
}
```

Explain contract

0 Listen on all transactions Filter wi

CALL [call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4 to: Counter.get() data: 0x6d4

CONCLUSION:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.